

Inference-cancelling transformer

Date: 2025-08-29, 2025-09-02, 2025-09-10, 2025-10-23, 2025-10-27, 2025-10-29, 2025-10-30, 2025-10-31, 2025-11-04, split to two directions on 2025-11-27, major revision 2025-11-28, then minor 2026-01-03

Author: Da-shan Shiu

Message: why the split

At some point I realized that if I kept analyzing the conventional transformer, I would have forever been limited by its inherent mysterious design. I can instead invent a new kind of transformer that maximally demystifies things. But, doing that involves a lot of risk. Therefore I decided to split Conjecture 5 to two directions. This one continues on adding minor changes to a conventional transformer to achieve the behavior of interference cancellation. A different paper talks about the by-design-no-mystery transformer.

Message to the researchers

This is a research that fundamental advances the theory underlying a transformer, *inspired* by known theories in cellular communications. But, worry not; you don't have to possess much knowledge about those theories, unless you want to know how I came up with the idea.

I think I had configured this research to a level where it can be executed by college seniors guided by an experienced researcher. The prerequisite for the college seniors is the ability to make small yet correct modification to transformer models and training codes, using a profiler to speed up training, applying disciplined training babysitting and in-training evaluation, and inquisitiveness plus perseverance to understand problems without making brash adjustments. Obviously, I shall provide almost all the details, so that my guys can proceed without too much guessing or too much exploration.

I believe that the idea will not work initially, but will eventually work after we discovered all the walls and solved them all with proper attention to details. Part of the fun of doing research is to be the first wave of people to hit those walls! My expectation is that despite we take the right directions and the best practices, it would still be difficult to pose a completion date with any confidence. Therefore, I would like to give this research to someone who wants to maximize learning while under very little pressure to meet a particular publication deadline.

To approach blocking issues, you shall de-sensitive your attention from end-to-end behaviors. Instead, insert "probes" into the model training pipeline, observe, and make supported understandings of why things behave the way they do. It is far more efficient to improve something when there is deep understanding; I do not encourage those "this seems like will improve end-to-end behavior so let's try that" efforts. This work is as much an "understanding a transformer" work as an actual interference cancellation attempt.

About the "low pressure" nature of this research: my goal is not necessarily that the shown architecture will be adopted as is in a SOTA transformer. My goal is to inspire ML practitioners to understand and apply interference cancellation mechanism inside a neural network. This research is not tied to a particular conference submission schedule.

Abstract

(Already rewritten. Now looks nice.) A transformer takes an input token, looks up past key-value histories, and propose an output distribution. The output distribution contains several viable continuation possibilities. Through an external "output token selection" mechanism, an output token is selected, and thus eliminating many of those possibilities. When this transformer proceeds to generate future tokens, even though it is aware of the only continuation that survived, the past key-value histories still contain all of the viable continuations. In other words, the "hindsight" was not used to refine the past histories to be as much consistent with the known generated token sequences as possible. We refer these *once plausible, turn impossible* token continuations as interference. In this research, via a few carefully crafted prompts, we quantify the implications of leaving and removing interference in the key-value histories over a few samples. We observe that there could be further room for improvement. With the understanding, we proceed to make architectural modifications to a transformer so the hindsight can be employed to reduce interference before proceeding with a next token generation. Our ablation study further informs us the individual contribution of the various steps in our interference-cancelling transformer.

1. Introduction

Write about transformer being great. (To be written.)

Write about the problem: transformer does not update KV to reflect selected tokens. (Copy from Abstract. A transformer takes an input token, looks up past key-value histories, and propose an output distribution. The output distribution contains several viable continuation possibilities. Through an external "output token selection" mechanism, an output token is selected, and thus eliminating many of those possibilities. When this transformer proceeds to generate future tokens, even though it is aware of the only continuation that survived, the past key-value histories still contain all of the viable continuations. We refer these *once plausible, turn impossible* token continuations as interference.)

Write about the problem more, especially mention any prior attempts found in the literature. Related ones. (In this research, via a few carefully crafted prompts, we quantify the implications of leaving and removing interference in the key-value histories over a few samples.) (The best practice in a communication detector is to, instead of diminishing interference by filtering, proactively estimate the value of interference and then cancel it by subtraction). (In other words, the "hindsight" was not used to refine the past histories to be as much consistent with the known generated token sequences as possible.)

Write about pre-solution quantification of problem and likely benefits. (We will first study the attention mechanism in a transformer and make an analogy that it can be thought of as applying a just-in-time filters on past signals. Just-in-time meaning that the filter coefficients are derived only immediately before the filters are to be applied. While these filters can attenuate conflicting information, as is shown in standard signal processing literature, this kind of "multiply and add" filters only have a limited interference rejection capability. instead of diminishing interference by filtering, proactively estimate the value of interference and then cancel it by subtraction. In our toy problems, we observe that there could be further room for improvement.)

Write about the solution linearly. (With the understanding, we proceed to make architectural modifications to a transformer to mitigate interference. These proposed modifications are collectively referred to as interference cancellation. ... hindsight can be employed... the transformer revises the KV cache before proceeding with the next token transformation... to reduce interference retrieved back from the attention mechanism ... enabled by corresponding modifications in training procedures...)

Write about the evaluations and results. (We show that interference cancellation meaningfully move the Pareto boundary of performance vs. model complexity. Our ablation study further informs us the individual contribution of the various steps in our interference-cancelling transformer.)

The contribution of this work includes:

- To be written
- ...
- To be written

Our goal of this work is (not necessarily that the proposed architecture modifications ... will be adopted as is in a SOTA transformer. My goal is to inspire ML practitioners to understand and apply interference cancellation mechanism inside a neural network.)

2. Background and related works

(Note to researchers: I think once you understand the concept, you can use a chatbot to improve these background items.)

2.1 Other types of transformer than the decoder-only type

This is just for completeness sake that there are other transformer types in addition to the decoder-only one. I myself can recall a Huggingface paper in the past that trained 3 types of transformer to compare their performances. I think Chatbot can find those various major kinds of transformers.

The purpose of this background item is to limit the scope of this paper to decoder-only transformer. Encoder-only, encoder-decoder, are excluded.

2.2 A Transformer alternates between transforming a latent state and selecting a token for input

Explains that a transformer "transforms" an input vector that encodes the identity of the token selected for the previous time step, to an output vector from which the token probability can be extracted by applying a learned linear projection.

Thus, the way a transformer works, it alternates between **transforming** an input vector to an output probability, and **selecting** a token as the input to the next round of transforming. The selection can be based on the output token distribution while at the same time taking into account of various penalties. In teacher forcing, the selection is independent of the output token distribution.

This alternate between two drastically different part is a "feature" for **autoregressive** models like a transformer. Diffusion and flow models generate the entire signal in one shot; they do not experience any such discontinuous state shift.

2.3 Attention mechanism as a just-in-time signal boosting operation

Here, I will try to frame attention as a form of signal boosting. The typical mechanism of a boost, is to selectively elevate the weight of desirable information. After boosting, the desirable signal's contrast with not-a-signal is enhanced. An explicit or implicit normalization is employed to keep numerical range back into control. Then more rounds of boosting (followed by normalization) can be done, eventually raising signal above noise.

In a transformer, there are two levels where this mechanism can take place.

Level 1.

At a given time step after layer l , the latent is converted to key and value and written into the KV cache as a KV tuple, (k, v) . Both the key and the value contains both information consistent with and inconsistent with the selected token at the top of a time step.

Currently the architecture misses the opportunity to boost the signal content of the KV cache.

Level 2.

The filter coefficient to selectively pass through signal is the product between the query q and the past $k_{<t}$: $u = q^\dagger k'$ normalized. The filter coefficient is formed at run time.

To filter, one passes the output of the first level filters $v'_{<t}$ through this filter.

The overall filter output is a (somewhat weirdly) weighted average of the retrieved values $v'_{<t}$ with the weights $u_{<t}$. Due to the filter, signal that are aligned with q are preferentially selected. This content is added back to the skip connection, thus boosting the signal content.

2.4 Interpreting and peeking at the latent space of transformers

In this paper, we base our intuition on an assumption that the latent h_{lt} carries information about plausible future continuations. Especially if a transformer is specifically trained that way; see references [Fast Inference from Transformers via Speculative Decoding], [Medusa], and [Better & Faster Large Language Models via Multi-token Prediction]. This deserves a bigger treatment, which will eventually become an appendix. See the next section.

Mechanisms such as output projection, Medusa heads, Gemma Scope, are meant to "peek" at some information hidden in a latent. However, they are learned, not by construction. But, at this moment, I do not have a constructive formulation that maps a prospective token sequence and its representation to within a latent at layer l at step τ .

Layer by layer, via attention and via FF, a transformer improves such belief. Gemini scope, etc., can be referenced to support the incremental improvement idea. Note that current literature of Gemini scope might only scope the present token. That can be extended to scope out future tokens.

2.5 Interference cancellation in communication and in signal processing

Say something about interference cancellation in communication and in signal processing.

The attention operation is like finding reference in a library: to find out the immediate word to continue a prompt, you go to the library, find all the books with the prompt in it, find out all the continuation in these books, do proper weighting, and use those information to help you decide which word to use. That's attention. I don't see an easy way to bring back "don't follow with this

word" from a library.

3 Signal, interference, and noise

In this section, we define the concept of signal, interference, and noise *given* hindsight.

3.1 Does a transformer latent carry information about prior and future tokens?

(**Note to researcher.** It is a common misconception that the latent for token t only contains information for the token to be selected at time t . This is a subsection designed to debunk that. Where to put this subsection is a problem left for later. Ideally, it can be placed as part of Background work. So, this section is not written in serious quality.)

You might ask the following question. If the training loss is just the next token loss, why would it contain all the future continuations? Of course, there are (though not 100% convincing) evidences suggesting just that. I simply accept that a transformer, in order to optimize the overall next token loss, learns to also predict future token and to store those information in the KV to be retrieved by self-attention. Because "it helps".

Although it is a bad argument, the taken-as-truth statement that a transformer is trained to just predict the current token is outdated. Many transformers are now trained to predict not just the present token but also future tokens. Therefore, practically speaking, we might as well assume that, starting from now, all good transformers will be trained to predict future continuations beyond the immediate next token. That becomes the "new normal" for LLM.

This does beget the question: to what extent quantitatively would learning to predict future tokens make the overall training objective better? Is there a math model? How do we optimize that? That is an interesting problem of itself that I do not touch here.

3.2 An illustrative example

For ease of expose, we describe the content of a token without the comma and the trailing space in the bodytext.

(**Message to researchers**

My assertion is that **the latent space of a transformer contains the belief of the transformer for all the past, current, and future token sequences, and their respective probabilities in some way, to inform subsequent generations via the attention mechanism, despite that a transformer is trained to predict just the immediate token.**)

(For the visually dominated folks, we can use this diagram. I am not sure it's necessary once this is in a final paper form though.)

```
<System Prompt> You are an assistant. You should respect the retrieved facts 100%.
<Retrieved Facts> There are three major ways to enhance health. Fasting, exercise, sleep.
There are three major ways to enhance health. Exercise, sleep, fasting. There are three
major ways to enhance health. Sleep, fasting, exercise. There are three major ways to
enhance health. Sleep, exercise, fasting. There are three major ways to enhance health.
Exercise, fasting, sleep. There are three major ways to enhance health. Fasting, sleep,
exercise. </Retrieved Facts> What are the three major ways to enhance health? Answer:
the three major ways to enhance health are
```

Table 1. An example of equally plausible continuations at a given point of token generation. We note that it is common that an English word, a trailing comma, and a trailing space, are together grouped as one token. For ease of expose, we describe the content of a token without the comma and the trailing space in the bodytext.

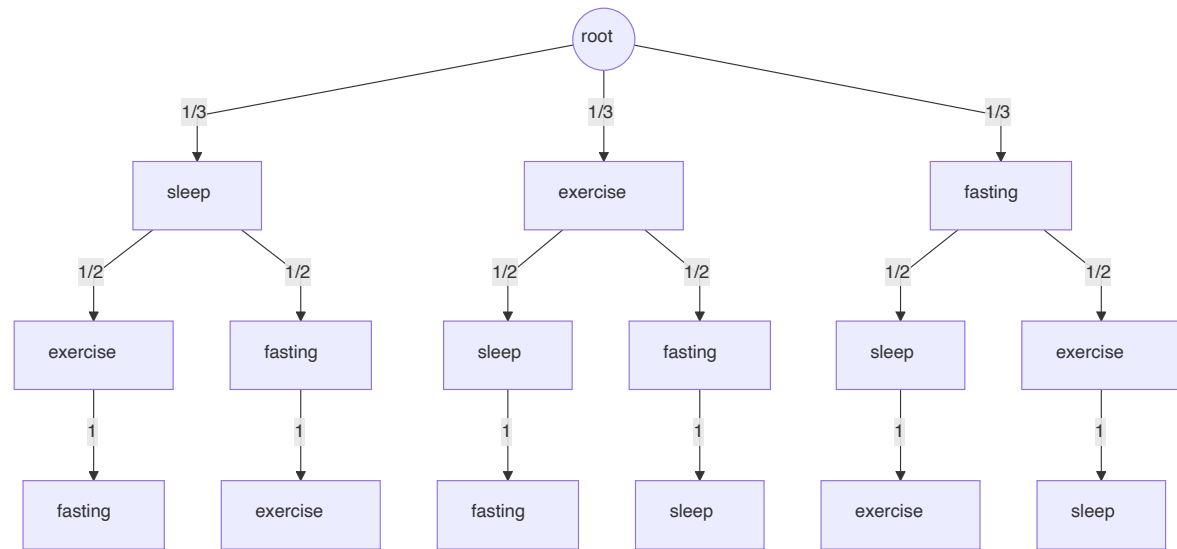


Figure 1. An example of three equally plausible continuations at a given point of token generation.

To introduce the concept of interference and interference cancellation given hindsight, consider the example prompt given in Table 1. At the time step (s) to generate the first output token, we realize that the intention of the carefully prepared prompt is to "mandate" that the future continuations and their respective probabilities are as shown by the tree in Figure 1. In Figure 1, from the root of this tree, there are three diverging branches, each of probability $1/3$. For each of the three Level-1 node, there are two diverging paths, each of probability $1/2$. For each of the six Level-2 node, there is only one path, leading to six Level-3 nodes.

Ideally, a transformer that strictly follows the instruction would have formulated internally this tree as well. If that is the case, the output probabilities at time s given by this transformer, should be $1/3$ for the three prospective tokens "sleep", "exercise", "fasting" and zero otherwise. Suppose at time step s the token "fasting," is chosen. That means the sampling process goes down the root-to-fastening path. During the next time step (t), the output probabilities of the transformer should be $1/2$ for the two prospective tokens "exercise" and "sleep" and zero otherwise.

(Seemingly redundant paragraph.) In this example, one uses the prompt given in Table 1. The last token, "are_", enters as an input token to a transformer. During this step (step t), from layer 1 to layer L , the latent is progressively being refined. Ideally, at the end of this step, the latent at the output layer (h_{Lt}) shall mean that all six possible continuations (sfe, sef, fse, fes, esf, efs) have the same probability $1/6$. When the output probability projection is applied, the output probabilities for these three tokens that can be selected for step t , "fasting,", "exercise,", and "sleep,", are all $1/3$. This probability distribution is submitted to the external "output token selection" mechanism.

Suppose "fasting," is selected. That also implies that "exercise,", and "sleep," are not selected.

(Note to researchers. I suggest that we actually do an experiment like this, to see if a typical instruct-LLM behaves in this way. This also serves as an encouragement to understanding the internal working of black-box neural networks.)

The concept of interference given hindsight can now be illustrated by analyzing the attention operation from the target time step t to the source time step s . At time step t , the token "fasting,_" is entered as the input token of the transformer. During the transforming phase of a transformer, the attention mechanism looks up the KV cache of preceding time steps to fetch useful information. If we zero in on the KV for time step s , we see that the KV shall contain information for all six equally probable continuations (sfe, sef, fse, fes, esf, efs). With the hindsight that "fasting" is selected, it is obvious that two of them - fse, fes - are consistent with the hindsight while the other four - sfe, sef, esf, efs - are not. We consider the information consistent with the hindsight as "signal", and the inconsistent as "interference". The information that are not related to any of the six are "noise".

(Seemingly redundant writing. The above is what we get from a conventional transformer. But, is this what we should accept? Given that we before time step $t + 1$ looks up the KV cache, we already know "fasting,_" is already selected, ideally we would have liked to benefit from the hindsight: for the KV cache of time step t to be corrected, that is, to contain only information that is consistent with the selected token, before time step $t + 1$ looks up the KV cache.)

To assess the potentially negative influence of such interference, one might contrast the situations against the case using the prompt in Table 2.

<System Prompt> You are an assistant. You should respect the retrieved facts 100%.

<Retrieved Facts> There are three major ways to enhance health. Fasting, exercise, sleep. There are three major ways to enhance health. Fasting, sleep, exercise. There are three major ways to enhance health. Fasting, exercise, sleep. There are three major ways to enhance health. Fasting, sleep, exercise. There are three major ways to enhance health. Fasting, sleep, exercise. There are three major ways to enhance health. Fasting, exercise, sleep. </Retrieved Facts> What are the three major ways to enhance health? Answer: the three major ways to enhance health are

Table 2. An example of a prompt to experimentally demonstrate the effect of ideal interference cancellation.

In this alternative case, for time step s , we apply teacher forcing so token "fasting" is selected. At time step t , when the latent in time step s is attended to, one sees that the other four continuations - sef, sfe, esf, efs - have probability zero. By attending to such past latent that are strictly consistent with hindsight, one expect that the behavior of a transformer to improve.

(Note to researchers. I suggest that we actually do an experiment like this, to see if a typical instruct-LLM behaves in this way. This also serves as an encouragement to understanding the internal working of black-box neural networks.)

3.3 Operational (qualitative) definition for signal, interference, and noise

We used example 1 to illustrate what is signal, interference, and noise contained in the latent, and example 2 to illustrate the behavior after applying ideal interference cancellation, viewed from the future with hindsight. We note that the effect of interference cancellation in Example 2 is achieved by editing a transformer's context buffer to be consistent with the actual token selection, followed by performing inference from token 0 again with teacher forcing. When

possible, in later sections we will refer to this edit-context method as a reference, even if it is somewhat onerous. In Section 4, we will propose alternative, more practical, methods.

(I think these are bad paragraphs. I don't need them. I only need to make my students understand my point.

Define signal, noise, and interference symbolically.

Do I need these definitions here?

Define signal, noise, and interference operationally.

The structure of the latent space of a transformer is not designed in a way that signal, interference, and noise, can be dis-entangled. Therefore it is not a guarantee that we can formulate latent $h = h_{\text{signal}} + h_{\text{interference}} + \text{noise}$. But still we can define an operational signal-to-interference ratio as,

signal = $f_{s,t}(h_s, x_t = \text{"fasting"})$ = unnormalized log prob of $x_t = \text{"fasting"}$, interference = $f_{s,t}(h_s, x_t = \text{"sleep"})$ = unnormalized log prob of $x_t = \text{"sleep"}$, signal / interference ratio = $\exp(\text{signal} - \text{sum of interference})$.

In (EQ), $f_{s,t}$ is a learned "scope" that produces from a latent h_s the unnormalized log probability it assigns to token t . An example of such a scope is Gemma Scope.

3.4 How the benefits of interference cancellation manifests itself

Argue that one can have more accurate output probabilities with interference cancellation, or one can use fewer layers to reach the same quality.

To argue the "fewer layer", reference the paper that made the latent convergence study.

4 Estimating and bounding the interference levels and the interference cancelling benefits

This is the domain for me. Students tend to be interested in actually working stuff. So students proceed.

5 Interference cancelling schemes

(Reminder. Interference cancellation with hindsight, when done right, removes or at least reduces the inconsistent component in K and V before K and V are referenced to by attention. How the benefits manifest, could be better performance, but could also be faster training, faster inference, smaller model, etc.)

The terminology.

- $\hat{x}_s$ is the token selected at *source* time s . $\neg\hat{x}_s$ is the token that did not get selected at time s .
- k_s and v_s is the original, as is, key and value for time s . k'_s and v'_s are the interference-adjusted versions, respectively.

- q_t is the original, as is, query from *target* time t .

5.1 Methods that do not require architecture modification

To study with vs. without interference cancellation, instead of training models that can do interference cancellation, I suggest that we first use smart prompts to observe the internal mechanisms and the subsequent benefits. (So called training-free methods.) Only when if we suspect that the model that we experiment with do not follow instructions well, do we post-train the model with instruction templates.

5.1.1 Prompt-based methods

Programmatically, form a prompt the following way. This should be part of the system prompt or the user prompt.

When you see the exact text "[SOME EXACT TEXT]", you must immediately continue with "[SOME OTHER TEXT]". Do not add any other words, explanations, or modifications. Simply output the continuation exactly as specified.

To see how to use this prompt, you can set

[SOME EXACT TEXT] to "What are the three major ways to enhance health? Answer:"

[SOME OTHER TEXT] to "fasting,_"

and then

[SOME EXACT TEXT] to "What are the three major ways to enhance health? Answer: fasting,_"

[SOME OTHER TEXT] to "sleep,_"

This way it allows you to perform interference cancellation from hindsight on a token-by-token basis. That is, you give this prompt, the chatbot answers with one token, you stop the chatbot, you reformulate the prompt to give the chatbot, it goes on until the chatbot finishes.

To contrast "no cancellation" vs. "with cancellation", you can put in entirely irrelevant texts as "no cancellation", and the above as "with cancellation".

We can extend this to

[SOME EXACT TEXT] to "What are the three major ways to enhance health? Answer:"

with

[SOME OTHER TEXT] to "fasting,_sleep, _"

If you think token-by-token is too slow, and you only do interference cancellation in a batched fashion.

Trick: to do the above experiment, use a tokenization visualizer to make sure that you cut right at a token-to-token border.

Trick: instead of the above health related text, if you want to observe the attention layer further, we can use rarely occurring sentences to kind of disable the FF layer from participating. Like, "a child humms dee bah duh dee bah dah".

5.1.2 Observing this prompt-based method

It's like if you ask a high school student to sum two integers. You can give them hints. The hints don't make them sum faster; though may allow them to sum faster.

You start with a good enough transformer. You also validate that such a transformer can faithfully execute what you want even without interference cancellation. Therefore, its external performance does not get improved by this training-free method. Instead, the benefits can manifest somewhere else.

Some of the common suspects.

- Improvement in attention patterns. With interference cancellation, the quality of the keys should be improved. As a result, the QK match *may* improve. You can observe the shift in attention weights. (I don't think this is a guarantee though because the model was not trained for this.)
- Improvement in number of layers to converge. With interference cancellation, the quality of the values should be improved. As a result, the feedback of V *may* improve the latent with bigger steps. You can observe the improvement in the convergence pattern of the latent from layer to layer. (Again, I don't think this is a guarantee though because the model was not trained for this.)

If you can think of other, let me know.

Both of these should lead to, if we train a model, a smaller (shallower) model than the one you experiment with.

Both of the should also lead to, if we ask the model a more difficult question, interference cancellation may help it to answer correctly.

5.1.3 Resources and Codes

You should get Antigravity and Gemini 3 Pro. We will reimburse you.

You should get a personal computing device and install a local git service to connect to MediaTek Research's private github.

You should run inference on a decently sized model using a plain vanilla approach. You must be able to dump the latents from the inference program. Your modification to the code should be checked in so every one can critique and can reproduce.

You shall use general purpose mathematical packages such as numpy or matlab to make the observation. Your scripts should be checked in so every one can critique and can reproduce.

Up to this point it can be manual. Manual meaning you "code" with the chatbot every step of the way.

If I am happy with your results, then it's scaling time.

1. You should run inference on a decently sized model using a professional approach. You must be able to dump the latent from the inference program. You should do what you need to do to make the speed far higher than before. Your modification to the code should be checked in.
2. You shall use general purpose mathematical packages such as numpy or matlab to make the observation. You should do what you need to do to make the speed far higher than before. Your scripts should be checked in.
3. You shall use general purpose package to convert your results into visual form.

Then you should iterate 1. 2. 3. with *no human in the loop*, making iterative improvements, until you are happy with the final run. Make sure that other people can independently reproduce your results.

5.2 Methods requiring architecture-modification

Because we deal with a modified transformer architecture that takes advantage of the hindsight to remove (some of) the interference in the latent.

5.2.1 Overview of a interference-cancelling transformer

During target time step t , the transformer network has the k and v from a source time step s . It also has both the selected token x_s and the most probably token that is not selected $\neg x_s$ as hindsight (side) information.

The candidate architecture I give here can be an overly luxurious example, reflecting my thought on 2025-10-23. Will it work? I don't know.

This architecture keeps the transformer almost the same as is; it only adds (interleaves) an additional interference cancellation step after the token is selected. That is, it adds a step to form

- $k'_s = K_{ic}(k_s, \hat{x}_s, \neg \hat{x}_s)$
- $v'_s = V_{ic}(v_s, \hat{x}_s, \neg \hat{x}_s)$

for some well-chosen mappings K_{ic} and V_{ic} . Giving these two mappings $\hat{x}_t$ and $\neg \hat{x}_t$, the purpose is to cancel as much the part in k_s (v_s) that aligns $\neg \hat{x}_t$ while keeping the part aligned with $\hat{x}_t$ as intact as possible.

This formulation can be expanded to cover deeper-in-the-future hindsight with $t > s$:

- $k'_s = K_{ic,t-s}(k_s, \hat{x}_t, \neg \hat{x}_t)$
- $v'_s = V_{ic,t-s}(v_s, \hat{x}_t, \neg \hat{x}_t)$

This expansion allows the correction of the past latent s that is a lot earlier than t . Which could be useful if you observe that attention patterns can look up tokens way in the past.

Similarly, in addition to cancel interference on a per-generated-token basis, we can do that in a batched fashion.

I expect that K_{ic} and V_{ic} shall be learned. If we start with an already trained model, we can keep all the original weights frozen, use some additional data to learn K_{ic} and V_{ic} , and aside from the architecture of K_{ic} and V_{ic} , that's pretty much it.

The architecture starts optionally with a projection that maps the last layer latent y_{t-1} to something you want to start with, which we denote as h_t^0 . We run an attention operation using queries from h_t^0 , x , $\hat{x}$, to get back a suggested improvement term that contains much less influence from $\hat{x}$. The output of this network is logits.

Due to K_{ic} and V_{ic} we will have more weights compared to the original transformer, but we will have more performance as well. So what's the point? I suppose that we don't ask that question yet. We will find out *new* behavior, the new-ness is itself value. We don't need to have utility in today's benchmarks and applications.

5.2.2 Crash course for the students

For the simplest case of received signal $x = s + i$, that the signal and interference are entangled in the most trivial manner.

The classical options are:

- use a linear operator to project closer to the signal and away from the interference. Of course in this case you don't know about the signal. You might know *something* about the signal, such as $\mathbb{E}[|x|^2]$. You also might know only *something* about the interference, such as $\mathbb{E}[|i|^2]$. Or even $\mathbb{E}[\text{Re}(xi^\dagger)]$. Solution in textbook. The cleaned up signal $\tilde{x} = Lx$ where L is (dynamically) derived from the known properties of the signal and the interference. Sometimes this is called interference mitigation, not cancellation.
- Use a subtraction-ish operator to obtain a cleaner signal as if the interference is directly reduced. This is often only applicable if you really know about the interference well.

In the situation you are dealing with now, armed with hindsight, you know about the interference perfectly. Unfortunately, the latent representation can be a messy entanglement, not a straightforward $x = s + i$. So designing a proper interference remover takes both intelligence and luck. In the future, you may not have the opportunities to work on such problems too often.

5.2.3 Useful reference: architecture for a speculative decoder

The Medusa speculative decoder uses one decoder head for the current token, another decoder head for the next token, and so on. For deep-in-the-future tokens, the hindsight knowledge of speculatively selected earlier tokens is not employed.

One architecture used is to linearly project the latent to a learned space, linearly project the last selected token to a learned space, concatenate the two representations, and pass over a learned MLP to form an estimate of the next token's output latent. And so on.

The two learned projections and the learned MLP are trained by using a proper multi-token training loss during pretraining.

5.2.4 Form of K_{ic} and V_{ic}

One architecture used is to linearly project a latent to a learned space, linearly project the last selected token to a learned space, linearly project the last most-prominent-yet-not-selected token to a learned space, concatenate the three representations, and pass over a learned MLP to form an estimate of the correction term to the latent.

5.2.5 Training

The entire computational graph is layer by layer, left to right. There is no recursion (no lower layer taking input from higher layer). Therefore, I don't anticipate any surprise or even much slowdown.

As you can see below in the next subsection, this is not the case for the more dramatic changes.

Here now is the summary on how to distill the interference-cancelling consistent transformer.

1. Prepare the teacher. Add the interference cancelling layer on top of the teacher transformer. Train to enhance the signal-to-interference probability ratio.
2. Distill into the student consistent transformer.
 - student's input at time $t+1$ = teacher's mitigated output at time t , potentially with a little additive noise.

- student's $\hat{x}_t$ and $\neg\hat{x}_t$: the selected token for the teacher at time t , and the highest probability token not selected by the teacher at time t .
- student's target unmitigated output at time $t+1$ = teacher's unmitigated output at time $t+1$
- student's target mitigated output at time $t+1$ = teacher's mitigated output at time $t+1$
- distillation loss: cross entropy wrt ideal unmitigated output and ideal mitigated output

3. Student in inference

- An appropriate $\hat{x}_t$ is injected at $t = 0$.
- Once the unmitigated output probability is produced, send the probability to the (external) token selection function, to return $\hat{x}_t$ and $\neg\hat{x}_t$.
- Progress through the interference cancelling layer, obtain the mitigated output.
- Feed the mitigated output as the input at the next time instant.

5.4 Quantities to monitor for trained interference cancelling transformer

We can monitor the change of latent from one layer to the next. See references in which the latent makes big jump at initial layers and final layers. By eliminating interference, the trajectory of latent evolution may change significantly.

6 Experiments and Results

to be written.

7 Conclusions

Cut and paste the four exit strategies I gave in the loop over here, starting from a multi-token training objective, to more effective speculative decoding scheme, to a bona fide better transformer, and finally to a new kind of transformer that takes the last layer latent as input and the selected token, dominant unselected tokens, and optionally their probabilities, as input. This last holy grail can be trained in a flow matching manner, but it's extremely optional.

References

@misc{wu2019deepgenerativemarkovstate,
 title={Deep Generative Markov State Models},
 author={Hao Wu and Andreas Mardt and Luca Pasquali and Frank Noe},
 year={2019},
 eprint={1805.07601},
 archivePrefix={arXiv},
 primaryClass={stat.ML},

```

url={https://arxiv.org/abs/1805.07601},
}

@misc{leviathan2023fastinference,
  title={Fast Inference from Transformers via Speculative Decoding},
  author={Yaniv Leviathan and Matan Kalman and Yossi Matias},
  year={2023},
  eprint={2211.17192},
  archivePrefix={arXiv},
  primaryClass={cs.LG},
  url={https://arxiv.org/abs/2211.17192},
}

@misc{cai2024medusa,
  title={Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads},
  author={Tianle Cai and Yuhong Li and Zhengyang Geng and Hongwu Peng and Jason D. Lee
and Deming Chen and Tri Dao},
  year={2024},
  eprint={2401.10774},
  archivePrefix={arXiv},
  primaryClass={cs.LG},
  url={https://arxiv.org/abs/2401.10774},
}

@misc{gloeckle2024betterfasterlarge,
  title={Better & Faster Large Language Models via Multi-token Prediction},
  author={Fabian Gloeckle and Badr Youbi Idrissi and Baptiste Rozière and David Lopez-Paz and
Gabriel Synnaeve},
  year={2024},
  eprint={2404.19737},
  archivePrefix={arXiv},
  primaryClass={cs.CL},
  url={https://arxiv.org/abs/2404.19737},
}

```

Out of scope at the moment but worthwhile to keep track of

4 Estimating and bounding the interference levels and the interference cancelling benefits

We will propose an operational definition for "signal", "interference", and "noise" in Section 4.2. Here, we go over our motivational example to make our points more concrete.

(

The time step τ is the step when "are" is fed into the transformer, and the time step t is the next time step. Here we will illustrate signal, interference, and noise from the output layer latent h_τ , given that token eventually selected is $\hat{y}^{\tau}$ is "sleep".

Let h_τ denote the latent representation at step τ . At a future time step $t > \tau$, this latent is consulted on how best to continue on generation. Qualitatively, **signal** is defined to be the part of h_τ that **agrees** with the selected token sequences from τ to $t - 1$, $\hat{y}_\tau^{t-1}$. **Interference** is defined to be the part of h_τ that **agrees** with other high probability token sequences that differ from the selected token sequence in at least one position. **Noise** is defined to be the part of h_τ that corresponds with neither.

)

Overview of what this section talks about. (Regardless, there must be a limit where the attention mechanism can fetch and can distinguish information. So, the information that is fetched back by an attention mechanism, will contain interference consistent with tokens that are not selected. Therefore, we expect that the latent of the next layer will contain some possibilities of selected token t being "exercise,_" and "sleeping,_" . With that, the attention mechanism of the next layer might also fetch back more interference, and so on.

I call these inconsistent information "interference". The question that we seek to answer is how much interference is there, how much and what kind of gain can we obtain if we can actively remove the interference, and if the gain is worthwhile to pursue, how should we do that.)

4.1 Incomplete rejection of interference

Do the math and show that there is incomplete rejection of interference. In the attention weights due to QK, and in the value V. (To be written)

Make remark that operationally we will focus on the interference contained in K and V. We will not talk about interference contained in Q. (From the above subsection, we know that the current "attention" mechanism operates more like adding the desired component and "normalize away" undesirable ones. There are two problems:

1. The returned value (V) can contain interference (marginal vs conditional).
2. Interference leaks into the query Q, which then responds with a key K that naturally contains interference.

To be written)

(May extract math from this paragraph; may be useful for Section 5 as Boosting-based pushing down of interference with everything else

As is explained in Section 2, the attention mechanism of a transformer appears to not naturally support a subtraction operator. Here I describe some approaches that, via boosting the signal with hindsight, enhances the signal to noise ratio.

Scheme 001. In this scheme, we make sure that a pedagogical transformer can handle multi-token conditioning. At $t = 0$, before we start, we are in a state called s_0 . As usual, we condition on the previous token x_0 . Eventually, we arrive at the selected token $\hat{x}_1$. Then we return back to state s_0 , condition on the previous token x_0 and the current token $\hat{x}_1$, go through the entire process, advance the transformer state to s_1 , without selecting a new token.

What is the intuition behind this? Well, consider the toy problem of sleep, exercise, and fast. When conditioning on the immediate past token x_0 which is "are", the latent space of the transformer is built up with equal part of sef, sfe, esf, ifs, fse, fes. If "sleep" is selected, we "ideally" would have preferred the KV cache to contain only the consistent proposals, sef and sfe. On the other hand, when conditioning on the immediate past token x_0 to be "are" and *speculatively* conditioning on the to-be-selected token to be "sleep", it makes intuitive sense that the latent built up would be exactly sef and sfe.

In summary, we run a batch size = 2 inference, to do a batch size = 1 inference with interference cancellation.)

4.2 Estimating and bounding the interference levels

(I think I will describe a hands-on experiment that can be carried out by a student. The student is expected to improve the experiments from this draft. Then, that student can rewrite this section. I will try not to **require** training a Gemma Scope.)

Whether there is significant gain to be had from interference cancellation depends on the amount of interference contained in the latent. To know that we need to first measure the amount of interference.

The first goal of this section is to review methods that might be considered for measuring the signal level and the (separate) interference level(s) in normal operations of a transformer. (We ... might attempt to fit the signal to noise level in a mathematical model.)

4.3 Estimating and bounding the interference level and the negative impacts from interference

The amount of interference that can be removed without disturbing the signal too much, how the benefits can be manifested and measured, and bound them

(Whether there is significant gain to be had from interference cancellation depends also on the amount of interference that can be removed while keeping the degradation of the signal minimal. To know that we

4.4 Some results

Write only if I have results (to be written)

4.5 Implication to the limit of speculative decoding performance

At step t , the step after "are_", during the entire step, all the latents are developed to reach the 1/3, 1/3, 1/3 among "fasting,_", "exercise,_" and "sleep,_" . If the LLM is any good, it should internally develop all six possibilities, and assign a probability to each one of them.

You can now see why speculative decoding has a limit. When there are legitimate reasons to have multiple plausible continuations, speculative decoding is predicted here to, simply, not work at all. But, speculative decoding is just a niche acceleration scheme; as long as for the majority of tokens in the generated sequence that such decoding works, that's good enough isn't it.

This is about using flow matching ... it can be a nice result, but I'd like to not assume that it is, unless and until I can prove that the transforming process reflects an ODE in the latent space as well. At this time I will just keep this content here.

5.2 Architecture of an interference-cancelling consistent-transformer

Here, we go for a very ambitious deviation from the existing transformer architecture, to combine interference cancellation with other things we had been trying to do in the past.

A consistent transformer is one in which the latent representation is *consistent* across all layers. For people first heard of this concept, consider a diffusion model that does outpointing.

Say the left side of an image is already known. A diffusion model is applied to paint to the right in a multi-turn manner, somehow one column of pixels per turn. Throughout the entire process, no matter it's the n -th turn, column c , row r , the definition of the value of a pixel is always the same. (E.g. R, G, B intensity). However, this is not true for a decoder-only transformer. There is no guarantee that the meaning for an n -th level, t -th step latent are the same across n and t . It's actually very easy to verify; at the input to layer 0, we put in the embedding of token $\hat{x}_{t-1}$, which should mean the probability of the prior token being $\hat{x}_{t-1}$ is 1. However, if at the output of layer L the latent is exactly this value, it means the probability of the to be selected token is to be computed by sending this vector to an output projection, softmax, temperature, etc... sampling process. Same value, different meaning. This difference can complicate the process of augmenting a transformer with interference cancellation, so we shall first make the modifications for a *consistent* transformer first.

The architectural candidate that I describe here, represents something in my mind on 2025-10-23. I can change my mind at any later time though.

The network consists of two parts.

1. The incremental developer. It takes in an initial draft of continuation probabilities, and outputs a vector that represents the final draft. It sound like a diffusion/flow matching model is suitable for this job. The latents are *consistent* In this incremental developer.
2. The cross-encoder. It takes the refined continuation probabilities for step t , the selected token, and the unselected token(s) as input, and outputs the draft continuation probabilities for step $t + 1$. It appears that an attention-only (no feedforward) transformer is a good choice for this task. The latents are *not consistent* In this cross-encoder.

5.2.1 The incremental developer: details

The incremental developer takes a form of a regular transformer, all the way down to positional embedding and all, except that it does not take in an embedding, but a "draft initial latent".

For whom can accept analogy only; if you are not someone who can take words literally, if you are the ones who ask "are we doing a diffusion model or a transformer", please skip every one of my analogy. My analogies can confuse you. How do you train a diffusion model? You start with ideal output, make corrupted versions, and learn how to recover. The idea of an incremental developer is similar: you can train one if you have the ideal output probabilities. You can disturb the ideal

output, and try to recover. How well it works in reality does depend on whether the mechanism of corruption reflects the reality.

The incremental developer takes in a vector and outputs a vector. The input vector is a corrupted version of the output. The draft is not only noisy but also has limited visibility to the more distant future. The knowledge encoded in the FF coefficients and the attention coefficients shall enable the improvement process to not only refine the current token's probability but also extends the prediction to one more token to the future. With a layered architecture, when attending to the past KV, one can choose to attend to the same layer, or attend to the final layer. The latter might be better for inference while the former might be easier for training.

Finally, to re-emphasize the "consistent" term: the same output head can be connected to any layer, to sample the output token. The "semantics" of the various layer latents are consistent.

5.2.2 The cross-encoder: details

The cross-encoder does two things: it strip away information consistent with unselected tokens, and it converts the latent from describing the probabilities of tokens start at time t to describing the probabilities of tokens start at time $t + 1$. The latter is why I call it "cross".

At the moment, there are quite a lot of freedom to design the cross encoder. I encourage you to propose new architectures based on your unique insights. Here, I will cite DeepSeek's multi-token predictor architecture as follows.

Start from the latent that feeds the external token selection module.

- The latent vector is concatenated with the embedding of $\hat{x}_t$ and the embedding of $\neg\hat{x}_t$, to form the input to a layer. Individual linear transformations can be applied before the concatenation to project to more advantageous processing spaces.
- Conducts a subtractive attention on the above concatenated vector, attending to past layers that are parallel to (or lower than) this one.
- FF layers are unlikely to be needed because we are only seeking to strip off information consistent with $\neg\hat{x}_t$ without hurting $\hat{x}_t$; we are not improving the information related to either, which is what the FF layers would do.
- Stack (i.e. repeat) a few such layers if necessary.

At the end of the interference cancelling layer, the major interference is hopefully mitigated. The desired signal is likely also hurt, but overall the signal-to-interference ratio should improve.

Figure X illustrates the consistent transformer. By incorporating the output of the sampler, the cross encoder converts the output latent for the previous time step into a draft latent for the incremental developer.

5.2.3 Training

At the moment I propose two approaches.

- A conventional option is to maximize the next token prediction probability at the output of the incremental developer. The advantage of this is simplicity.
- A radical option is to form a flow matching loss for the incremental developer; that is, the training loss is a combination of next token prediction loss observed from each layer of the incremental developer. The advantage of this is the possibility to instantiate the incremental developer by flow matching, which offers the opportunity to reduce layer

count and to increase weight reuse.

Initialize the cross encoder and the incremental developer somehow (see below). To build one batch of training data, we first run the current generation model to obtain the last layer latent at the output of the incremental developer. We use the recorded last layer latent as the input to the forward propagation. We record the gradient via backward propagation. In either approaches, we apply teacher forcing for both $\hat{x}_t$ and $\neg\hat{x}_t$. The next token loss can be replaced by a distillation loss to speed up training, if an oracle model exists.

Will this work? Arguably, yes or no; in the above, I did not use back propagate in time, which was a necessity for recursive neural network (RNN).

If the above works (maybe with some hacks), then the training time should be only slightly longer than conventional transformer training.

This might be where we hit the biggest uncertainty, the biggest test for our brain power!

5.2.4 weight initialization

I am tempted to do the following initialization.

- for the incremental improvement, build a dataset that the output is either the inverse of 1 hot probability or ideal latent from a strong model, and the input to be disturbed version of the ideal output. By how much to disturb is a hyper parameter;
- for the cross encoder, apply the same logic;

We train the two segments separately, and use the result as initialization.

This might be another area where we hit a significant uncertainty. But somehow I think we can deal with this one just with perserverence.

4.4 Interference cancelling attention mechanism

The above motivates us to wonder, what if a transformer can not only welcome signal that is consistent with the selected token, but also reject *known* interference signals?

In other words, if $\tilde{x}_t$ is one known strong candidate token, $\tilde{\tilde{x}}_t$ is another known strong candidate token, we envision an interference cancelling transform executing a type of attention that looks like

$$K', V' = K, V - \text{softmax}(\tilde{x}_t Q K) K, V$$

$$K'', V'' = K', V' - \text{softmax}(\tilde{\tilde{x}}_t Q K') K', V'$$

$$z_t = \text{softmax}(x_t Q K'') V'' .$$

I pick the output level representation for the m -th turn, m being large enough, sans any positional embedding, to be the shared **representational format**. I then inject this into the transformer as the input to layer 0 for the $m + 1$ -th turn. As long as I keep the *output head weight* constant over all values of m , this makes at least the input representations and the output representations consistent-in-format. For step $t + 1$, each transformer layer is additionally informed of the embeddings of the selected $\hat{x}_t$ and one unselected $\neg\hat{x}_t$. We can modify the attention mechanism and the FF mechanism in many ways to accommodate $\hat{x}_t$ and one unselected $\neg\hat{x}_t$, which we will go over later. With the output head weight frozen, the entire

transformer is trained to predict in a denoised distillation manner from a teacher model:

- student's input at time $t+1$ = teacher's output at time t , potentially with a little additive noise
- student's $\hat{x}_t$ and $\neg\hat{x}_t$: the selected token for the teacher at time t , and the highest probability token not selected by the teacher at time t .
- student's target output at time $t+1$ = teacher's output at time $t+1$
- distillation loss: cross entropy wrt ideal output

This architecture is a natural one later on when we want to replace a fixed-number-of-layer transformer to one that can vary its number of layers by means of flow matching training. We will get to that when we are there.

Now that we have a consistent transformer, how do we inform it with the positive $\hat{x}_t$ and the interfering $\neg\hat{x}_t$? That is an awesome question.

-

===

Suppose that "fasting,_" is selected as the output token for step t . At step $t + 1$, the weakness of the attention mechanism shows up. For this transformer, only "fasting,_" is fed in as input. Therefore, in its attention mechanism, it can only additively accumulate from past KV the part of signal that is consistent with "fasting,_" being selected. However, an additive averaging $z_t = \text{softmax}(x_t QK)V$ process can and does bring back signal that correspond to "exercise,_" and "sleep,_" . What may make matters worse is that from layer to layer the interference from "exercise,_" and "sleep,_" can be highly aligned, meaning that it's not effective to average them out.

===

In short, from a signal processing perspective, z_t contains a signal component that we ideally want to fetch back, but also an interfering component that we do not want. Furthermore, unlike uncorrelated noise, it is not effective to rely on many independent averaging attempts (over past tokens and over layers) to suppress interference.

===

The classical procedure of transformer token selection at time step τ is output token probability = $\text{softmax}(W_{out}h_\tau)$.

Let me try a "micro" definition,

- signal: if we can decompose h_τ to be $h_1 + h_0$ so that the probability of the selected token is sensitive only to h_1 , then the signal level can be defined to be inversely related to the sensitivity of output probability with respect to a scaling factor a , if $h = ah_1 + h_0$. Please suggest how to make this decomposition, how to define and operationalize a , and how best to define the "inverse".
- interference: similar to the case of signal, but replace "selected" token to a particular competing token.
- noise: I don't know how to define, so I don't care.

The hindsight transformer

Consider this hypothetical situation. Let's say we allow a transformer to generate from scratch following the Start of sequence special token. After a few tokens, we find that the output of the model is "How now brown cow". Now we want to continue to generate using the same model. The question then becomes, what are the best latent values, for the first four tokens, and for all the layers, that can optimize the continuation loss, conditioned on the knowledge that the first four tokens generated are now known?

The optimal KV values post token selection are not the values prefore token selection.

Here I define the term "hindsight transformer". A "hindsight transformer" is a normal transformer with a "KV cache edit" functionality added behind the output token selection. For step t , the transformer behaves as usual until it presents an output probability for output token t . After a token decision is made, we now know that the output token is $\hat{x}_t$, the KV cache and the token $\hat{x}_t$ is sent into the "KV cache edit" function. This function revises the KV cache so that the training loss for future continuation is optimized under the new values .

How this KV cache edit function is made, how costly is it, I encourage you to not worry about them. Just appreciate the concept.

###

6 Experiments and Results

6.1 Application areas of the interference cancelling concept

6.1.1 As a multi-token training cost scheme

Copy and paste...

6.1.2 As a speculative decoding scheme

Copy and paste...

6.1.3 Deterministic tranformer's inference

Copy and paste...

6.1.4 Stochastic transformer's inference

Copy and paste...

additionally experiments with keeping the latent space over all the layers aligned. It is hypothesized that doing so would shed light on how to find a smaller network, e.g. one that uses fewer layers.

6.2 Results

Some ablation.

(We argue that appreciable gain is expected on the tokens following those tokens that dominant-probability token either does not exist or does not get selected.

If true, how to take advantage: if interference cancellation can be effective, I expect to see that the original transformer's output can be matched by one with fewer total number of layers, or the output probabilities to get closer to a teacher for those tokens.)

6.2.1 Definition of the computational complexity for training

The purpose of this subsection is to define the Pareto front between computational intensity and accuracy at training

6.2.2 Definition of the computational complexity for inference

The purpose of this subsection is to define the Pareto front between computational intensity and accuracy at inference.

6.2.3 As a multi-token training cost scheme

Copy and paste...

6.2.4 As a speculative decoding scheme

Copy and paste...

6.2.5 Deterministic transformer's inference

Copy and paste...

6.2.6 Stochastic transformer's inference

Copy and paste...

additionally experiments with keeping the latent space over all the layers aligned. It is hypothesized that doing so would shed light on how to find a smaller network, e.g. one that uses fewer layers.